

企业级 AI Agent 工作流平台

一、项目背景

随着大模型（LLM）的快速发展，企业越来越希望利用 AI 提升知识管理、文档处理、业务自动化等工作效率。

传统聊天机器人仅能完成简单问答，而企业场景往往需要：

- 知识库检索
- 多步骤任务规划
- 外部工具调用
- 人工审批
- 长期记忆管理
- 流程追踪与审计

因此，本项目设计并实现一个基于 LangChain 与 LangGraph 的企业级 AI Agent 工作流平台。

项目分为两个阶段：

第一阶段：LangChain 版本

实现标准 Agent 能力：

- 多轮对话
- 工具调用
- RAG知识库
- 会话记忆
- 流式输出

第二阶段：LangGraph 版本

实现生产级 Agent 工作流：

- 状态管理
- 工作流编排
- Checkpoint恢复
- Human-in-the-Loop
- 多Agent协作
- 长期记忆管理

通过两个版本的实现，展示从 Agent Prototype 到 Production Agent 的完整演进过程。

二、项目目标

构建一个企业知识工作流助手，帮助用户完成：

- 企业知识查询

- 文档分析
- 方案生成
- 数据查询
- 工单处理
- 流程审批

最终实现：

用户需求



任务规划



知识检索



工具调用



结果生成



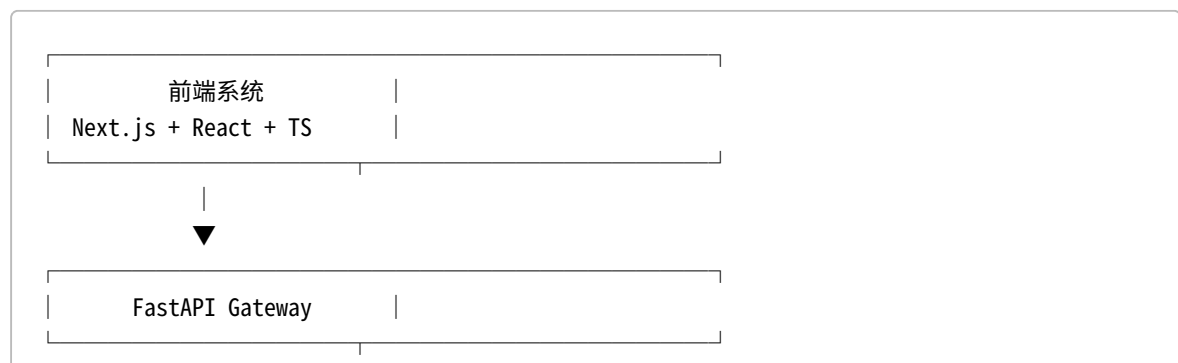
人工审批

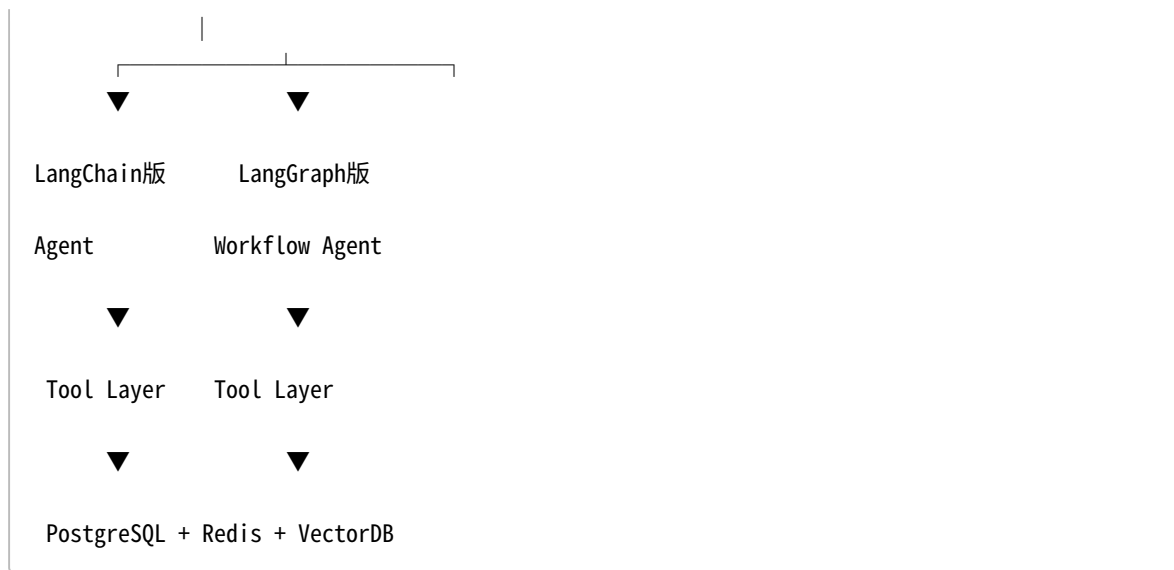


结果归档

三、系统架构

整体架构





四、核心功能设计

1. 智能对话

支持：

- 多轮上下文
- 流式回复
- Token统计
- 历史会话管理

示例：

用户：

"帮我生成一份数据库优化方案"

Agent：

1. 分析需求
2. 检索知识库
3. 查询案例
4. 输出优化方案

2. RAG知识库

支持：

文档上传

- PDF
- Word
- Markdown
- TXT

文档解析

- LangChain Loader

文本切分

- RecursiveCharacterTextSplitter

Embedding

可选：

- OpenAI Embedding
- BGE-M3
- Qwen Embedding

向量存储

开发版：

- Chroma

生产版：

- pgvector
-

3. Agent工具调用

内置工具：

搜索工具

```
@tool
def search_tool(query):
    pass
```

SQL查询工具

```
@tool
def query_db(sql):
    pass
```

文档分析工具

```
@tool
def analyze_doc(file):
    pass
```

Python执行工具

```
@tool
def run_python(code):
    pass
```

邮件发送工具

```
@tool
def send_email(content):
    pass
```

4. 会话记忆

LangChain版

ConversationBufferMemory

保存：

- 最近对话
- 用户偏好

LangGraph版

Memory Store

支持：

- 短期记忆
 - 长期记忆
 - 用户画像
-

5. 文件分析

用户上传：

- PDF

- Word
- Excel

Agent自动完成：

- 内容提取
- 摘要生成
- 风险分析
- 报告生成

6. workflow 执行

LangGraph版本新增

流程：

开始

↓

需求分析

↓

任务规划

↓

知识检索

↓

工具调用

↓

结果校验

↓

人工审批

↓

输出结果

↓

结束

五、LangChain实现方案

Agent架构

用户输入

↓

Prompt

↓

LLM

↓

Tool Calling

↓

Observation

↓

Final Answer

采用：

- LangChain Agent
- ReAct模式
- Tool Calling

技术特点：

- 开发快
- 上手简单
- 适合PoC验证

局限：

- 状态管理弱
- 流程复杂度有限

六、LangGraph实现方案

Graph结构

START

↓

Planner Node

↓

Retriever Node

↓

Executor Node

↓

Validator Node

↓

Human Review Node

↓

END

State定义

```
class AgentState(TypedDict):  
    question: str  
    plan: str  
    documents: list  
    tool_results: list  
    final_answer: str
```

Checkpoint机制

保存：

- 当前节点

- 当前状态
- 历史执行记录

支持：

- 中断恢复
- 失败重试
- 流程回放

Human-in-the-Loop

在关键节点：

审批通过

↓

继续执行

审批拒绝

↓

重新规划

应用场景：

- 合同审核
- 方案审批
- 财务流程

七、前端设计

技术栈：

- Next.js
- TypeScript
- TailwindCSS
- Shadcn UI

页面设计

首页

功能：

- Agent聊天窗口
- 文件上传
- 历史会话

workflow 监控页

展示:

Planner

✓

Retriever

✓

Executor

运行中

Validator

等待

实时显示:

- 当前节点
- Tool调用
- Token消耗
- 耗时统计

知识库管理页

支持:

- 上传文档
- 删除文档
- 重建索引
- 查看Embedding状态

八、数据库设计

users

id

username

email
created_at

conversations

id
user_id
title
created_at

messages

id
conversation_id
role
content
created_at

documents

id
filename
vector_status
created_at

workflow_logs

id
workflow_id
node_name
status
execution_time

九、项目亮点

技术亮点

LangChain Agent

展示：

- Tool Calling
 - RAG
 - Memory
-

LangGraph Workflow

展示：

- State Management
 - Workflow
 - Checkpoint
 - Human-in-the-Loop
-

LangSmith

展示：

- Tracing
 - Evaluation
 - Debugging
-

RAG系统

展示：

- Embedding
 - Retrieval
 - Rerank
-

工程化能力

展示：

- Docker部署
- Redis缓存
- PostgreSQL
- API设计

- 权限管理

十、面试亮点讲解

面试时重点说明：

为什么先做LangChain?

快速验证Agent能力。

适合：

- MVP
- 原型验证

为什么升级到LangGraph?

解决：

- 多步骤任务
- 状态管理
- 流程恢复
- 人工介入

问题。

LangGraph相比LangChain优势

能力	LangChain	LangGraph
Tool Calling	√	√
RAG	√	√
多步骤流程	一般	强
状态管理	弱	强
Checkpoint	×	√
人工审批	×	√
多Agent协作	一般	强

十一、未来优化方向

Multi-Agent

增加：

- Planner Agent
- Research Agent
- Coding Agent
- Review Agent

协同完成复杂任务。

MCP支持

接入：

- 数据库MCP
- GitHub MCP
- 企业系统MCP

实现标准化工具生态。

长期记忆系统

构建：

- 用户画像
 - 偏好学习
 - 个性化推荐
-

SaaS化部署

支持：

- 多租户
 - 企业知识库
 - 权限管理
 - Agent市场
-

十二、预期收获

通过本项目可以向面试官展示：

1. LLM应用开发能力
2. LangChain开发能力
3. LangGraph开发能力
4. RAG系统设计能力
5. Agent workflow设计能力
6. Python后端开发能力
7. 前后端联调能力
8. AI系统工程化能力

项目完成后可直接作为：

- Agent开发工程师
- AI应用开发工程师
- LLM工程师
- AI平台工程师

岗位面试项目。